

Lock Implementations: Lessons From Homework 3

Lecture 14

Professor Amanda Bienz

Ticket Lock Implementation

```
#include "src.hpp"

#define _GNU_SOURCE

void init(lock_t* lock)
{
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t* lock)
{
    int myturn = __sync_fetch_and_add(&(lock->ticket), 1);
    while (lock->turn != myturn)
    {
        // spin
    }
}

void unlock(lock_t* lock)
{
    __sync_fetch_and_add(&(lock->turn), 1);
}

void destroy(lock_t* lock)
{
}
}
```


Test Mutual Exclusion

```
#include "src.hpp"

lock_t my_lock;
volatile int value = 0;
int n_loops = 10000;

void* thread(void* arg)
{
    lock(&my_lock);
    for (int i = 0; i < n_loops; i++)
        value++;
    unlock(&my_lock);

    return NULL;
}

int main(int argc, char** argv)
{
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

```
TEST(TLBTest, TestsIntests)
{
    init(&my_lock);

    int n_threads = 10;
    pthread_t* threads = new pthread_t[n_threads];

    for (int i = 0; i < n_threads; i++)
    {
        pthread_create(&(threads[i]), NULL, thread, NULL);
    }

    for (int i = 0; i < n_threads; i++)
    {
        pthread_join(threads[i], NULL);
    }

    ASSERT_EQ(value, n_threads*n_loops);

    destroy(&my_lock);
}
```


Unit Test: Fails!

```
tests — -zsh — 72x33
...enchmarks/benchmarks/osu/osu-bw-timeout/bin — ssh lassen
...mework-3-bienz2/build/OSHomeworkSource/tests — -zsh +
[bienz@amandas-mbp tests % ./spin_mutual_exclusion
[=====] Running 1 test from 1 test suite.
[-----] Global test environment set-up.
[-----] 1 test from TLBTest
[ RUN      ] TLBTest.TestsIntests
/Users/bienz/OneDrive - University of New Mexico/Teaching/OperatingSystems/Homeworks/Homework 3/homework-3-bienz2/OSHomeworkSource/tests/test_mutual_exclusion.cpp:47: Failure
Expected equality of these values:
  value
    Which is: 65007
  n_threads*n_loops
    Which is: 100000
[ FAILED   ] TLBTest.TestsIntests (0 ms)
[-----] 1 test from TLBTest (0 ms total)

[-----] Global test environment tear-down
[=====] 1 test from 1 test suite ran. (0 ms total)
[ PASSED   ] 0 tests.
[ FAILED   ] 1 test, listed below:
[ FAILED   ] TLBTest.TestsIntests

1 FAILED TEST
bienz@amandas-mbp tests %
```


Can ChatGPT Solve this?

My Question:



ticket_spin_lock.cpp
File



test_mutual_exclusion.cpp
File

I have implemented a ticket spin lock and am testing mutual exclusion (both files attached). Why is this test failing?

Can ChatGPT Solve this?

The Response:

Your mutual exclusion test is failing because your `ticket_spin_lock` implementation does not enforce proper **compiler or memory ordering**. Although the ticketing logic correctly orders the threads logically, without memory fences, the compiler or CPU might reorder instructions such that updates to `value` occur outside the critical section or are visible too early/late to other threads.

What's missing?

Your `lock()` and `unlock()` functions lack **memory barriers**, which are needed to:

- prevent the compiler from hoisting `value++` out of the critical section
- ensure that memory writes inside the lock are visible to other threads only after the lock is acquired

Out-of-Order Execution

- **Out-of-order-execution:** Your compiler is not required to execute your code in the same order it is written.
- Example: Two independent operations such as below can happen in any order (or at the same time), you will still get a correct answer.

```
x = x + 5;  
y = y * 2;
```


Out-of-Order Execution

- Are my critical section and my unlock() method independent operations?

```
lock(&my_lock);  
for (int i = 0; i < n_loops; i++)  
    value++;  
unlock(&my_lock);
```

```
void unlock(lock_t* lock)  
{  
    __sync_fetch_and_add(&(lock->turn), 1);  
}
```

- The compiler does not know we will be using multiple threads, assumes these can be executed out of order.
- On some processors, atomic operations include a **memory barrier**, but not on all computers (e.g. ARM).

ChatGPT's Solution

- Include `__sync_synchronize`: enforces a memory barrier. Can be inside the *lock* and *unlock* functions, but let's add this into the mutual exclusion test to avoid changing your homework code.

```
void* thread(void* arg)
{
    lock(&my_lock);
    __sync_synchronize();
    for (int i = 0; i < n_loops; i++)
        value++;
    __sync_synchronize();
    unlock(&my_lock);

    return NULL;
}
```


Unit Test: Still fails...

```
tests — -zsh — 72x33
...enchmarks/benchmarks/osu/osu-bw-timeout/bin — ssh lassen
...mework-3-bienz2/build/OSHomeworkSource/tests — -zsh
+
bienz@amandas-mbp tests % ./spin_mutual_exclusion
[=====] Running 1 test from 1 test suite.
[-----] Global test environment set-up.
[-----] 1 test from TLBTest
[ RUN    ] TLBTest.TestsIntests
/Users/bienz/OneDrive - University of New Mexico/Teaching/OperatingSystems/Homeworks/Homework 3/homework-3-bienz2/OSHomeworkSource/tests/test_mutual_exclusion.cpp:49: Failure
Expected equality of these values:
  value
    Which is: 69613
  n_threads*n_loops
    Which is: 100000
[ FAILED ] TLBTest.TestsIntests (0 ms)
[-----] 1 test from TLBTest (0 ms total)

[-----] Global test environment tear-down
[=====] 1 test from 1 test suite ran. (0 ms total)
[ PASSED ] 0 tests.
[ FAILED ] 1 test, listed below:
[ FAILED ] TLBTest.TestsIntests

1 FAILED TEST
bienz@amandas-mbp tests %
```

Back to ChatGPT?

My Follow-up:



test_mutual_exclusion.cpp
File

I added `__sync_synchronize` to the unit test (included) and am using the same lock code as before. The unit test is still failing. Why?

Back to ChatGPT?

The Response:

Adding `__sync_synchronize()` inside the critical section (in the test code) doesn't fix the root problem because mutual exclusion must be enforced **before entering** and **after exiting** the critical section. In your code:

cpp

 Copy

 Edit

```
lock(&my_lock);  
__sync_synchronize(); // Too late  
for (...) value++;      // Already unsafe if lock failed  
__sync_synchronize(); // Too early to unlock  
unlock(&my_lock);
```

This doesn't help if `lock()` allows multiple threads in concurrently due to instruction reordering or missing memory barriers.

Back to ChatGPT?

The actual issue:

Your `ticket_spin_lock` implementation (unchanged from before) still lacks **memory barriers at the lock boundaries**, so it doesn't guarantee visibility and ordering. You **must add barriers inside the lock implementation**, not the user code.

Fix the lock implementation like this:

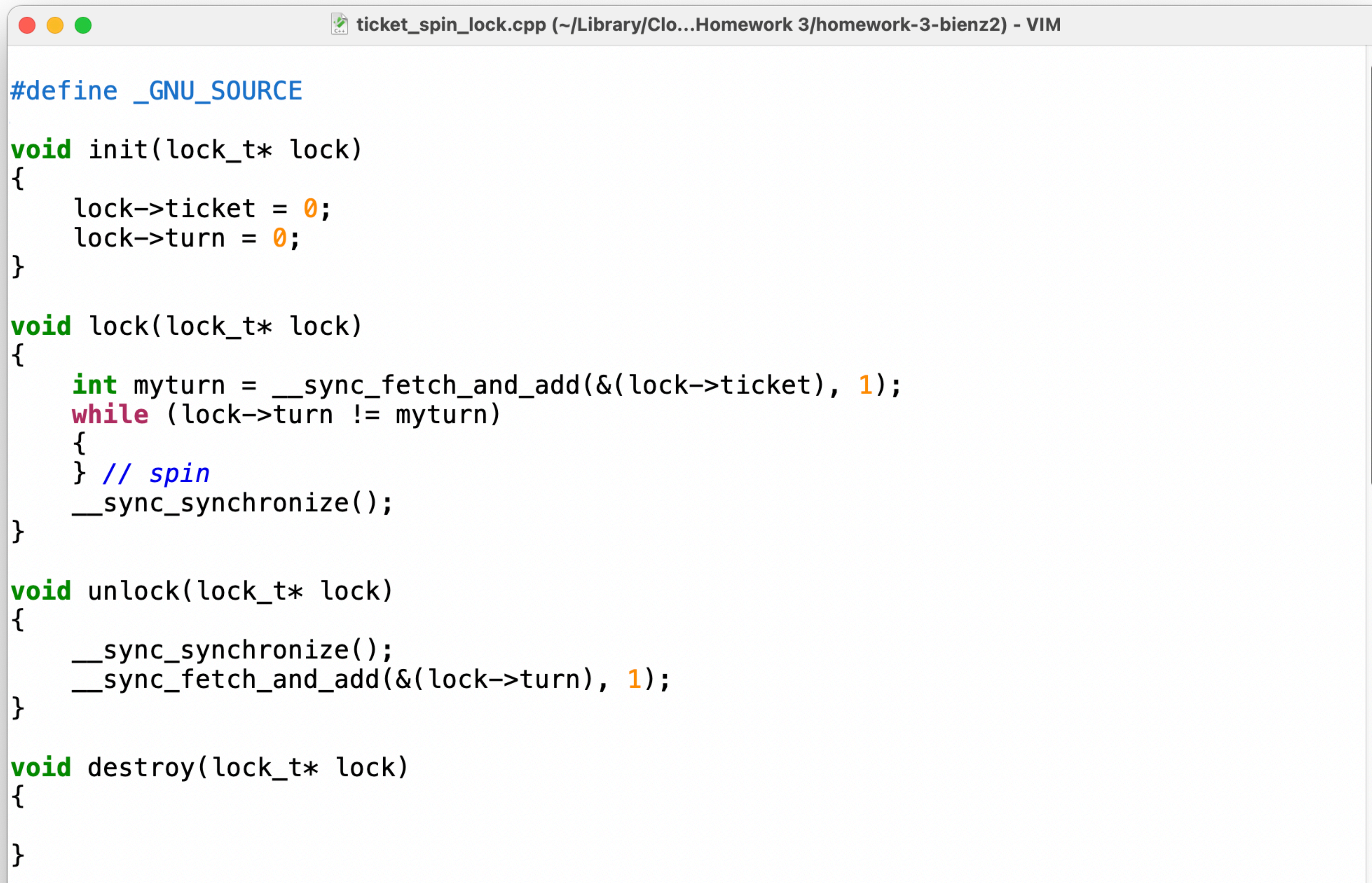
```
cpp                                                                    Copy Edit

void lock(lock_t* lock)
{
    int myturn = __sync_fetch_and_add(&(lock->ticket), 1);
    while (lock->turn != myturn)
    {
        // spin
    }
    __sync_synchronize(); // Acquire memory barrier
}

void unlock(lock_t* lock)
{
    __sync_synchronize(); // Release memory barrier
    __sync_fetch_and_add(&(lock->turn), 1);
}
```

My LLM is hallucinating...

In Case You're Curious...



```
ticket_spin_lock.cpp (~/.Library/Clo...Homework 3/homework-3-bien2) - VIM

#define _GNU_SOURCE

void init(lock_t* lock)
{
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t* lock)
{
    int myturn = __sync_fetch_and_add(&(lock->ticket), 1);
    while (lock->turn != myturn)
    {
        // spin
    }
    __sync_synchronize();
}

void unlock(lock_t* lock)
{
    __sync_synchronize();
    __sync_fetch_and_add(&(lock->turn), 1);
}

void destroy(lock_t* lock)
{
}
```

Back to ChatGPT?

My Follow-up:



ticket_spin_lock.cpp

File

I added the memory barriers into the lock as well. It is still failing the unit test. Why?

Back to ChatGPT?

You're very close! You've added memory fences correctly **inside the** `lock()` **and** `unlock()`, but the test still fails because of a **data race on** `value++` due to it being only `volatile` and not atomic.

The real issue now: `value++` **is not atomic.**

Even though your ticket lock **should** ensure mutual exclusion, the statement:

cpp

Copy

Edit

```
value++;
```

is not atomic. It's compiled roughly as:

cpp

Copy

Edit

```
int tmp = value;
tmp = tmp + 1;
value = tmp;
```

We have stumped ChatGPT...

And since `value` is `volatile int`, it does not guarantee atomicity or prevent reordering, only visibility. If **multiple threads enter the critical section** due to a lock bug, or if the compiler reorders, this will cause a race condition.

A Hint

- The tests are passing at a low compiler optimization level (-O0)
- They only fail when I include -O3
- **Let's check out assembly**

Assembly of Lock + Test

The image shows a screenshot of the Godbolt compiler explorer interface. The left pane displays C++ source code for a lock implementation, and the right pane shows the corresponding x86-64 assembly code generated by GCC 14.2.

C++ source code (left):

```

1 // Type your code here, or load an example.
2 #include <pthread.h>
3
4 #define GNU_SOURCE
5
6 // Lock Struct
7 typedef struct __lock_t
8 {
9     int ticket;
10    int turn;
11 } lock_t;
12
13 void init(lock_t* lock)
14 {
15     lock->ticket = 0;
16     lock->turn = 0;
17 }
18
19 void lock(lock_t* lock)
20 {
21     int myturn = __sync_fetch_and_add(&(lock->ticket), 1);
22     while (lock->turn != myturn)
23     {
24     } // spin
25     __sync_synchronize();
26 }
27
28 void unlock(lock_t* lock)
29 {
30     __sync_synchronize();
31     __sync_fetch_and_add(&(lock->turn), 1);
32 }

```

x86-64 gcc 14.2 (Editor #1) assembly output (right):

```

1 init(__lock_t*):
2     push    rbp
3     mov     rbp, rsp
4     mov     QWORD PTR [rbp-8], rdi
5     mov     rax, QWORD PTR [rbp-8]
6     mov     DWORD PTR [rax], 0
7     mov     rax, QWORD PTR [rbp-8]
8     mov     DWORD PTR [rax+4], 0
9     nop
10    pop     rbp
11    ret
12 lock(__lock_t*):
13    push    rbp
14    mov     rbp, rsp
15    mov     QWORD PTR [rbp-24], rdi
16    mov     rax, QWORD PTR [rbp-24]
17    mov     edx, 1
18    lock xadd     DWORD PTR [rax], edx
19    mov     DWORD PTR [rbp-4], edx
20    nop
21 .L3:
22    mov     rax, QWORD PTR [rbp-24]
23    mov     eax, DWORD PTR [rax+4]
24    cmp     DWORD PTR [rbp-4], eax
25    jne     .L3
26    lock or     QWORD PTR [rsp], 0
27    nop
28    pop     rbp
29    ret
30 unlock(__lock_t*):

```

Add -O3

The screenshot displays the Compiler Explorer web application. The left pane shows the C++ source code for a thread-local storage (TLS) implementation. The right pane shows the assembly output generated by x86-64 gcc 14.2 with the -O3 optimization level. The assembly code is color-coded to match the source code's logic.

C++ Source Code (Left Pane):

```
1 // Type your code here, or load an example.
2 #include <pthread.h>
3
4 #define GNU_SOURCE
5
6 // Lock Struct
7 typedef struct __lock_t
8 {
9     int ticket;
10    int turn;
11 } lock_t;
12
13 void init(lock_t* lock)
14 {
15     lock->ticket = 0;
16     lock->turn = 0;
17 }
18
19 void lock(lock_t* lock)
20 {
21     int myturn = __sync_fetch_and_add(&(lock->ticket), 1);
22     while (lock->turn != myturn)
23     {
24     } // spin
25     __sync_synchronize();
26 }
27
28 void unlock(lock_t* lock)
29 {
30     __sync_synchronize();
31     __sync_fetch_and_add(&(lock->turn), 1);
32 }
33
34 void destroy(lock_t* lock)
```

Assembly Output (Right Pane):

```
1 thread(void*):
2     lock add     DWORD PTR my_lock[rip], 1
3     lock or QWORD PTR [rsp], 0
4     mov     ecx, DWORD PTR n_loops[rip]
5     test    ecx, ecx
6     jle     .L2
7     xor     edx, edx
8 .L3:
9     mov     eax, DWORD PTR value[rip]
10    add     edx, 1
11    add     eax, 1
12    mov     DWORD PTR value[rip], eax
13    cmp     edx, ecx
14    je      .L2
15    jmp     .L3
16 thread(void*) (.cold):
17 .L2:
18     lock or QWORD PTR [rsp], 0
19     lock add     DWORD PTR my_lock[rip+4], 1
20 init(__lock_t*):
21     mov     QWORD PTR [rdi], 0
22     ret
23 lock(__lock_t*):
24     lock add     DWORD PTR [rdi], 1
25     lock or QWORD PTR [rsp], 0
26     ret
27 unlock(__lock_t*):
28     lock or QWORD PTR [rsp], 0
29     lock add     DWORD PTR [rdi+4], 1
30     ret
31 destroy(__lock_t*):
```


Found it!

- My while loop was optimized away...
- Threads don't spin... of course this breaks!

```
void lock(lock_t* lock)
{
    int myturn = __sync_fetch_and_add(&(lock->ticket), 1);
    while (lock->turn != myturn)
    {
        } // spin
    __sync_synchronize();
}
```

```
lock(__lock_t*):
    lock add     DWORD PTR [rdi], 1
    lock or QWORD PTR [rsp], 0
    ret
```

How do we fix this?

Volatile Keyword

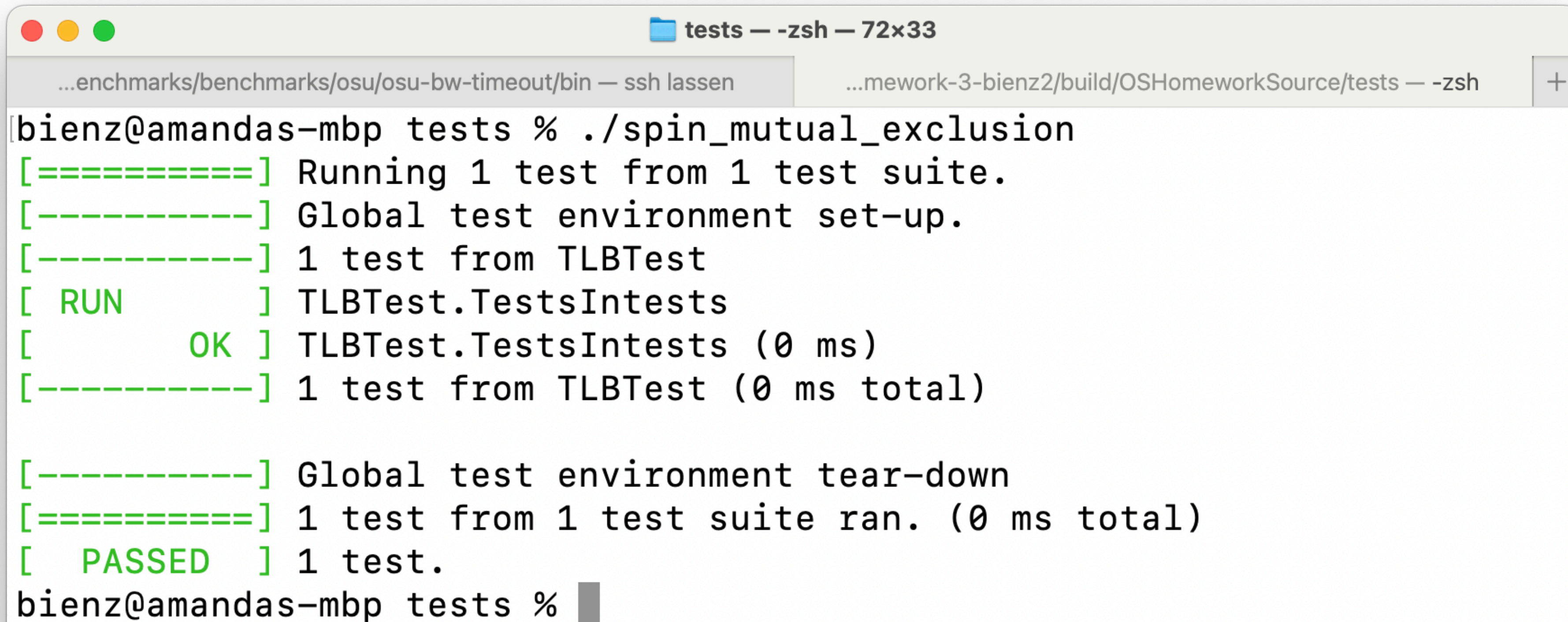
- All parts of our lock need to be volatile
- Now, compiler will not optimize those loops away

```
// Lock Struct
typedef struct __lock_t
{
    // Needed for ticket locks
    volatile int ticket;
    volatile int turn;

    // Needed for semaphore lock
    volatile int S;
    pthread_mutex_t mutex;

    // Needed for queue lock
    volatile int flag;
    volatile int guard;
    queue_t queue;
} lock_t;
```


Finally Passing!



```
tests — -zsh — 72x33
...enchmarks/benchmarks/osu/osu-bw-timeout/bin — ssh lassen
...mework-3-bienz2/build/OSHomeworkSource/tests — -zsh +
[bienz@amandas-mbp tests % ./spin_mutual_exclusion
[=====] Running 1 test from 1 test suite.
[-----] Global test environment set-up.
[-----] 1 test from TLBTest
[ RUN      ] TLBTest.TestsIntests
[          OK ] TLBTest.TestsIntests (0 ms)
[-----] 1 test from TLBTest (0 ms total)

[-----] Global test environment tear-down
[=====] 1 test from 1 test suite ran. (0 ms total)
[ PASSED   ] 1 test.
bienz@amandas-mbp tests %
```

Some Takeaways

- Writing working systems code using threads is **hard**
- Understanding how your compiler works is necessary
- You may need to read some assembly
- **ChatGPT is excellent when doing tasks you already understand.**